

A photograph of industrial machinery, likely a valve or pump, with a pressure gauge. The machinery is made of metal and shows signs of wear and rust. The background is a light-colored brick wall. The text "Machine Learning in Production Automating and Testing ML Pipelines" is overlaid on the image in white. The pressure gauge has a scale from 0 to 100 and the brand name "KRAVIZ" is visible. The valve has "150mm" stamped on it.

Machine Learning in Production Automating and Testing ML Pipelines

Infrastructure Quality...

Fundamentals of Engineering AI-Enabled Systems

Holistic system view: AI and non-AI components, pipelines, stakeholders, environment interactions, feedback loops

Requirements:

System and model goals
User requirements
Environment assumptions
Quality beyond accuracy
Measurement
Risk analysis
Planning for mistakes

Architecture + design:

Modeling tradeoffs
Deployment architecture
Data science pipelines
Telemetry, monitoring
Anticipating evolution
Big data processing
Human-AI design

Quality assurance:

Model testing
Data quality
QA automation
Testing in production
Infrastructure quality
Debugging

Operations:

Continuous deployment
Contin. experimentation
Configuration mgmt.
Monitoring
Versioning
Big data
DevOps, MLOps

Teams and process: Data science vs software eng. workflows, interdisciplinary teams, collaboration points, technical debt

Responsible AI Engineering

Provenance,
versioning,
reproducibility

Safety

Security and
privacy

Fairness

Interpretability
and explainability

Transparency
and trust

Ethics, governance, regulation, compliance, organizational culture

Readings

Required reading (accidentally assigned last week): Eric Breck, Shanqing Cai, Eric Nielsen, Michael Salib, D. Sculley. [The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction](#). Proceedings of IEEE Big Data (2017)

Learning Goals

- Decompose an ML pipeline into testable functions
- Implement and automate tests for all parts of the ML pipeline
- Understand testing opportunities beyond functional correctness
- Describe the different testing levels and testing opportunities at each level
- Automate test execution with continuous integration

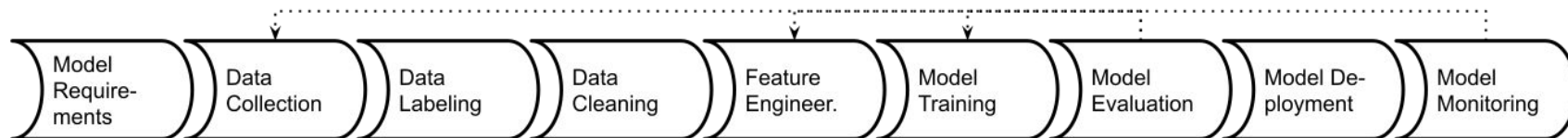
Example Repo

Runnable code versions of all examples in this lecture in this repo:

<https://github.com/mlip-cmu/pipeline-testing>

Feel free to follow along and experiment with it during the lecture

Thinking in Pipelines



Common ML Pipeline

```
# load data collected from team1
import pandas as pd

url = 'http://128.2.25.78:8080/private/log1.clean'
df = pd.read_csv(url)
df.head()
```

	dayIdx	user	userAvgTime	location	dow	isWeekend	time
0	0	Pittsburgh66Correy	7.045001	Pittsburgh	6	True	0.000000
1	1	Pittsburgh66Correy	7.045001	Pittsburgh	7	True	6.883333
2	2	Pittsburgh66Correy	7.045001	Pittsburgh	1	False	6.816667
3	3	Pittsburgh66Correy	7.045001	Pittsburgh	2	False	7.383333
4	4	Pittsburgh66Correy	7.045001	Pittsburgh	3	False	0.000000

Data was preprocessed externally, identifying the time at a given day when the light was first turned on (12pm). Weather and sunrise information is not included here, though that'd be important. If the light was on this morning (quite common), 0 is recorded.

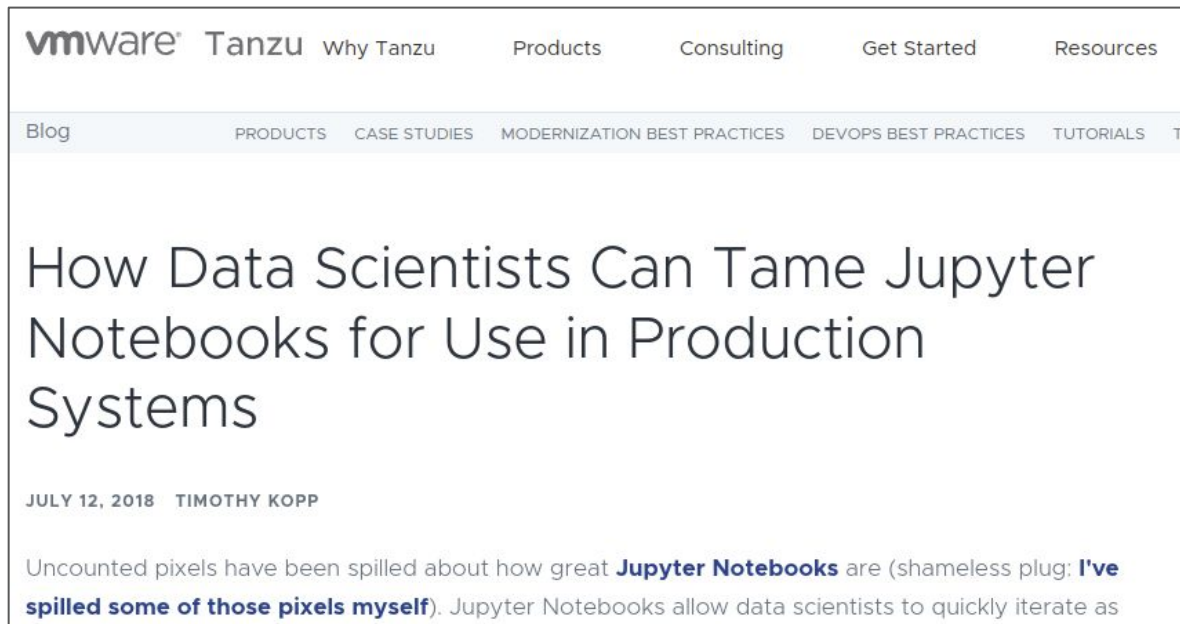
```
[ ] # just data encoding and splitting X and Y

X = df.drop(['time'], axis=1)
YnonZero = df['time'] > 0
Y = df['time']

from sklearn import preprocessing
# leDate = preprocessing.LabelEncoder()
# leDate.fit(X['date'])
# leDate.transform(X['date'])

X=X.apply(preprocessing.LabelEncoder().fit_transform)
X
```

Notebooks as Production Pipeline?



Parameterize and use nbconvert?

What's Wrong with Computational Notebooks? Pain Points, Needs, and Design Opportunities

Souti Chattopadhyay¹, Ishita Prasad², Austin Z. Henley³, Anita Sarma¹, Titus Barik²

Oregon State University¹, Microsoft², University of Tennessee-Knoxville³
{chattops, anita.sarma}@oregonstate.edu, {ishita.prasad, titus.barik}@microsoft.com, azh@utk.edu

ABSTRACT

Computational notebooks—such as Azure, Databricks, and Jupyter—are a popular, interactive paradigm for data scientists to author code, analyze data, and interleave visualizations, all within a single document. Nevertheless, as data scientists incorporate more of their activities into notebooks, they encounter unexpected difficulties, or pain points, that impact their productivity and disrupt their workflow. Through a systematic, mixed-methods study using semi-structured interviews ($n = 20$) and survey ($n = 156$) with data scientists, we catalog nine pain points when working with notebooks. Our findings suggest that data scientists face numerous pain points throughout the entire workflow—from setting up notebooks to deploying to production—across many notebook environments. Our data scientists report essential notebook requirements, such as supporting data exploration and visualization. The results of our study inform and inspire the design of computational notebooks.

Author Keywords

Computational notebooks; challenges; data science; interviews; pain points; survey

CCS Concepts

•Human-centered computing → Interactive systems and

Azure,¹ Databricks,² Colab,³ Jupyter,⁴ and nteract.⁵ While originally intended for exploring and constructing computational narratives [29, 31], data scientists are now increasingly orchestrating more of their activities within this paradigm [33]: through long-running statistical models, transforming data at scale, collaborating with others, and executing notebooks directly in production pipelines. But as data scientists try to do so, they encounter unexpected difficulties—pain points—from limitations in affordances and features in the notebooks, which impact their productivity and disrupt their workflow.

To investigate the pain points and needs of data scientists who work in computational notebooks, across multiple notebook environments, we conducted a systematic mixed-method study using field observations, semi-structured interviews, and a confirmation survey with data science practitioners. While prior work has studied specific facets of difficulties in notebooks [24, 17], such as versioning [18, 19] or cleaning unused code [13, 34], the central contribution of this paper is a taxonomy of validated pain points across data scientists' notebook activities.

Our findings identify that data scientists face considerable pain points through the entire analytics workflow—from setting up the notebook to deploying to production—across many notebook environments. While our participants reported

Subtle Bugs in Data Wrangling Code

```
df['Join_year'] = df.Joined.dropna().map(  
    lambda x: x.split(',')[1].split(' ')[1])
```

```
df.loc[idx_nan_age, 'Age'].loc[idx_nan_age] =  
    df['Title'].loc[idx_nan_age].map(map_means)
```

```
df["Weight"].astype(str).astype(int)
```

Subtle Bugs in Data Wrangling Code (continued)

```
df['Reviws'] = df['Reviews'].apply(int)
```

```
df["Release Clause"] =  
    df["Release Clause"].replace(regex=['k'], value='000')  
df["Release Clause"] =  
    df["Release Clause"].astype(str).astype(float)
```

Subtle Bugs in Data Wrangling Code (continued)

```
num = data.Size.replace(r'[kM]+$ ', '', regex=True).  
    astype(float)  
factor = data.Size.str.extract(r'[\d\.]+([KM]+)',  
                               expand=False)  
factor = factor.replace(['k', 'M'], [10**3, 10**6]).fillna(1)  
data['Size'] = num*factor.astype(int)
```

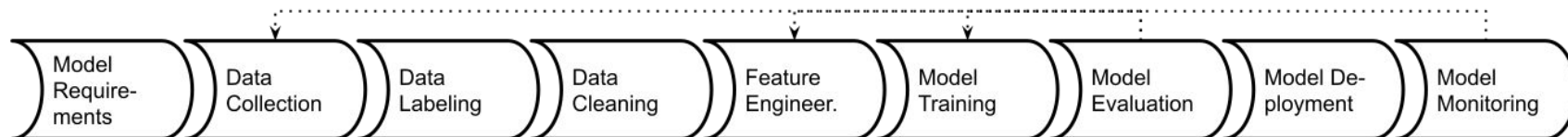
```
data["Size"] = data["Size"].  
    replace(regex=['k'], value='000')  
data["Size"] = data["Size"].  
    replace(regex=['M'], value='000000')  
data["Size"] = data["Size"].astype(str).astype(float)
```

Silent Mistakes

Mistakes in ML Pipelines usually do not *crash*, do not throw exceptions

Simply producing wrong outputs, poor models, missing updates

Possible Silent Mistakes in ML Pipelines? (in each stage)



Possible Mistakes in ML Pipelines

Danger of "silent" mistakes in many phases:

- Dropped data after format changes
- Failure to push updated model into production
- Incorrect feature extraction
- Use of stale dataset, wrong data source
- Data source no longer available (e.g web API)
- Telemetry server overloaded
- Negative feedback (telemtr.) no longer sent from app
- Use of old model learning code, stale hyperparameter
- Data format changes between ML pipeline steps

Pipeline Thinking

After exploration and prototyping build robust pipeline

One-off model creation -> repeatable automateable process

Enables updates, supports experimentation

Explicit interfaces with other parts of the system (data sources, labeling infrastructure, training infrastructure, deployment, ...)

Design for change

Building Robust Pipeline Automation

Support experimentation and evolution

- Automate

- Design for change

- Design for observability

- Testing the pipeline for robustness

Thinking in pipelines, not models

Integrating the Pipeline with other Components

Focus today: Testing the pipeline

We tested the model before (and it may still make mistakes)

Now we test the code around the model

Data wrangling, training, deployment, etc

Pipeline Testability and Modularity

Pipelines are Code

From experimental notebook code to production code

Each stage as a function or module

Well tested in isolation and together

Robust to changes in inputs (automatically adapt or crash, no silent mistakes)

Use good engineering practices (version control, documentation, testing, naming, code review)

Sequential Data Science Code in Notebooks

How to test??

```
# typical data science code from a notebook
df = pd.read_csv('data.csv', parse_dates=True)

# data cleaning
# ...

# feature engineering
df['month'] = pd.to_datetime(df['datetime']).dt.month
df['dayofweek'] = pd.to_datetime(df['datetime']).dt.dayofweek
df['hour'] = pd.to_datetime(df['datetime']).dt.hour
df['delivery_count'] = boxcox(df['delivery_count'], 0.4)
df.drop(['datetime'], axis=1, inplace=True)

dummies = pd.get_dummies(df, columns = ['month', 'weather', 'dayofweek', 'hour'])
dummies = dummies.drop(['month_1', 'hour_0', 'weather_1'], axis=1)

X = dummies.drop(['delivery_count'], axis=1)
y = pd.Series(df['delivery_count'])

X_train, X_test, y_train, y_test =
    train_test_split(X, y, test_size=0.3, random_state=1)

# training and evaluation
lr = LinearRegression()
lr.fit(X_train, y_train)

print(lr.score(X_train, y_train))
print(lr.score(X_test, y_test))
```

Testability can be decomposed into...

Controllability:

- Ability to influence system internal state or behavior by changing its inputs

Observability

- Degree to which you can determine that the behavior went as expected.

Testable Code

Think about testing when writing code

Unit testing encourages you to write testable code

Separate parts of the code to make them independently testable

Abstract functionality behind interface, make it replaceable

Bonus: Test-Driven Development is a design and development method in which you *always* write tests *before* writing code

Pipeline restructured into separate functions

```
def encode_day_of_week(df):
    if 'datetime' not in df.columns: raise ValueError("Column datetime missing")
    if not is_string_dtype(df.datetime): raise ValueError("Invalid type for column datetime")
    df['dayofweek'] = pd.to_datetime(df['datetime']).dt.day_name()
    df = pd.get_dummies(df, columns = ['dayofweek'])
    return df

# ...

def prepare_data(df):
    df = clean_data(df)
    df = encode_day_of_week(df)
    df = encode_month(df)
    df = encode_weather(df)
    df.drop(['datetime'], axis=1, inplace=True)
    return (df.drop(['delivery_count'], axis=1),
            encode_count(pd.Series(df['delivery_count'])))

def learn(X, y):
    lr = LinearRegression()
    lr.fit(X, y)
    return lr

def pipeline():
    train = pd.read_csv('train.csv', parse_dates=True)
    test = pd.read_csv('test.csv', parse_dates=True)
    X_train, y_train = prepare_data(train)
    X_test, y_test = prepare_data(test)
    model = learn(X_train, y_train)
    accuracy = eval(model, X_test, y_test)
    return model, accuracy
```

Orchestrating Functions

```
def pipeline():  
    train = pd.read_csv('train.csv', parse_dates=True)  
    test = pd.read_csv('test.csv', parse_dates=True)  
    X_train, y_train = prepare_data(train)  
    X_test, y_test = prepare_data(test)  
    model = learn(X_train, y_train)  
    accuracy = eval(model, X_test, y_test)  
    return model, accuracy
```

Dataflow frameworks like [Luigi](#), [DVC](#), [Airflow](#), [d6tflow](#), and [Ploomber](#) support distribution, fault tolerance, monitoring, ...

Hosted versions like [DataBricks](#) and [AWS SageMaker Pipelines](#)

Test the Modules

```
def encode_day_of_week(df):
    if 'datetime' not in df.columns: raise ValueError("Column datetime missing")
    if not is_string_dtype(df.datetime): raise ValueError("Invalid type for column datetime")
    df['dayofweek'] = pd.to_datetime(df['datetime']).dt.day_name()
    df = pd.get_dummies(df, columns = ['dayofweek'])
    return df
```

```
def test_day_of_week_encoding():
    df = pd.DataFrame({'datetime': ['2020-01-01', '2020-01-02', '2020-01-08'],
                       'delivery_count': [1, 2, 3]})
    encoded = encode_day_of_week(df)
    assert "dayofweek_Wednesday" in encoded.columns
    assert (encoded["dayofweek_Wednesday"] == [1, 0, 1]).all()

# more tests...
```

How to select test cases?



Modularity fosters Testability

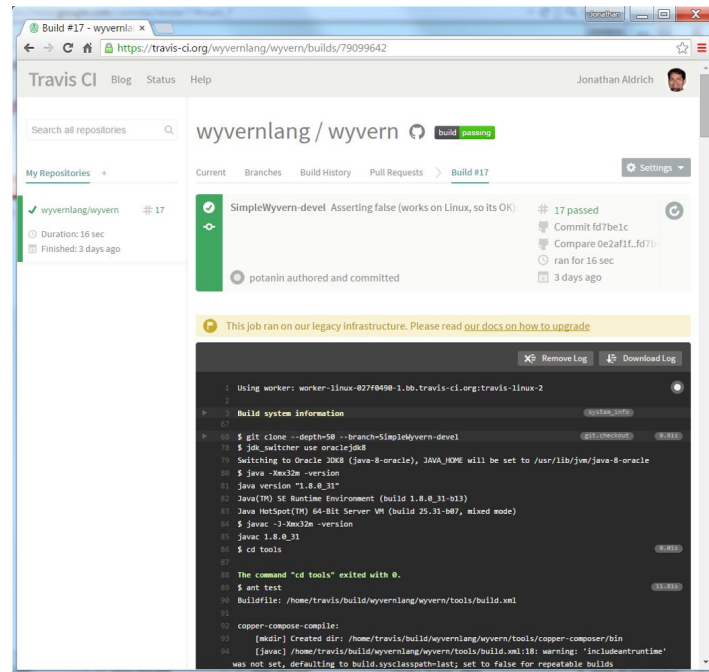
Breaking code into functions/modules

Supports reuse, separate development, and testing

Can test individual parts

Test Automation

From Manual Testing to Continuous Integration



Anatomy of a Unit Test

Specification

Controlled environment

Test inputs (calls and parameters)

Expected outputs/behavior (oracle)

```
import pandas as pd
import pytest
from pipeline import parse_size # code under test

# common reusable abstractions
@pytest.fixture
def app_sizes():
    return pd.Series(["19M", "201k", "2.5M",
                     "Varies with device"])

def test_parse_size_converts_units(app_sizes):
    """Sizes with k and M suffixes become bytes.
       Other values become NaN."""
    # run code under test
    result = parse_size(app_sizes)
    # check expected results (oracle)
    assert result[:3].tolist() == [19_000_000,
                                    201_000, 2_500_000]
    assert pd.isna(result[3])
```

Unit Testing Pitfalls

Working code, failing tests

"Works on my machine"

Flaky tests, ignored failed tests

Tests break frequently

How to avoid?

Build systems & Continuous Integration

Automate all build, analysis, test, and deployment steps from a command line call

Ensure all dependencies and configurations are defined

Ideally reproducible and incremental

Distribute work for large jobs

Track results

Key CI benefit: Tests are regularly executed, part of process

Tracking Model Qualities

More on that in a later
MLOps lecture

mlflowGithub Docs

Listing Price Prediction

Experiment ID: 0 Artifact Location: /Users/matei/mlflow/demo/mlruns/0

Search Runs: Search

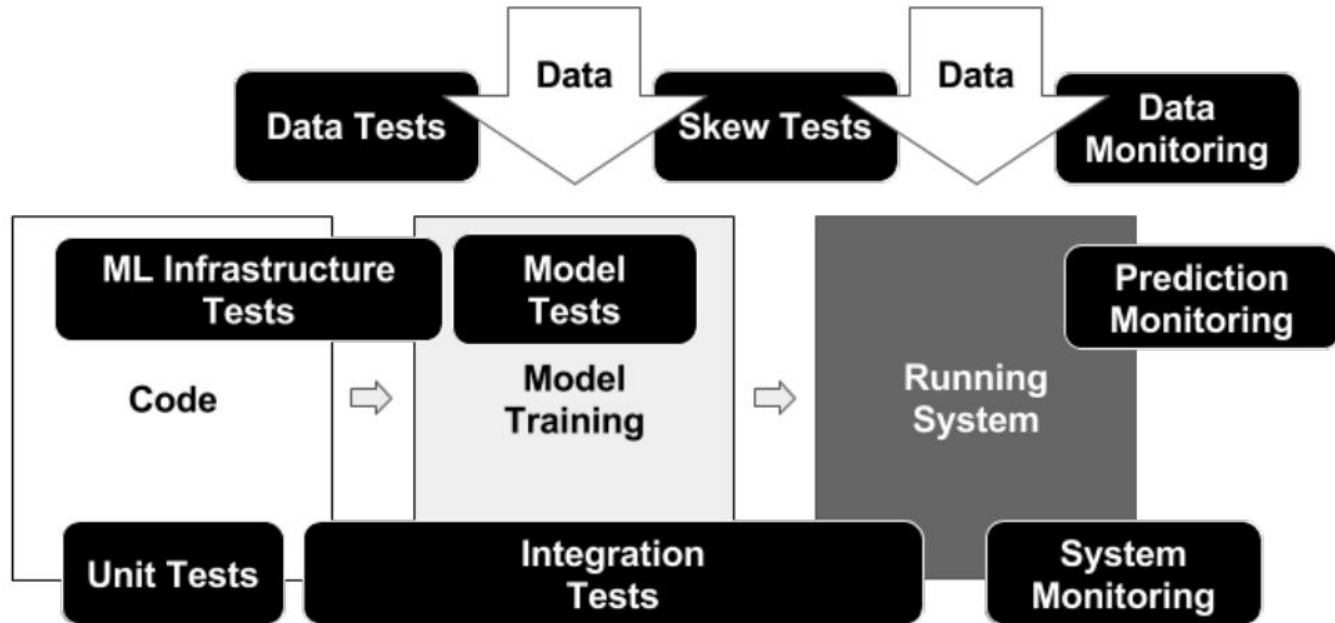
Filter Params: Filter Metrics: Clear

4 matching runs Compare Selected Download CSV

	Time	User	Source	Version	Parameters		Metrics		
					alpha	l1_ratio	MAE	R2	RMSE
☐	17:37	matei	linear.py	3a1995	0.5	0.2	84.27	0.277	158.1
☐	17:37	matei	linear.py	3a1995	0.2	0.5	84.08	0.264	159.6
☐	17:37	matei	linear.py	3a1995	0.5	0.5	84.12	0.272	158.6
☐	17:37	matei	linear.py	3a1995	0	0	84.49	0.249	161.2

Many tools: MLFlow, Neptune, TensorBoard, Weights & Biases, Comet.ml, ...

Testing Maturity



Case Study: Covid-19 Detection



(from S20 midterm; assume model deployment in the cloud)

Data Tests



1. Feature expectations are captured in a schema.
2. All features are beneficial.
3. No feature's cost is too much.
4. Features adhere to meta-level requirements.
5. The data pipeline has appropriate privacy controls.
6. New features can be added quickly.
7. All input feature code is tested.

Tests for Model Development



1. Model specs are reviewed and submitted.
2. Offline and online metrics correlate.
3. All hyperparameters have been tuned.
4. The impact of model staleness is known.
5. A simpler model is not better.
6. Model quality is sufficient on important data slices.
7. The model is tested for considerations of inclusion.

ML Infrastructure Tests



1. Training is reproducible.
2. Model specs are unit tested.
3. The ML pipeline is Integration tested.
4. Model quality is validated before serving.
5. The model is debuggable.
6. Models are canaried before serving.
7. Serving models can be rolled back.

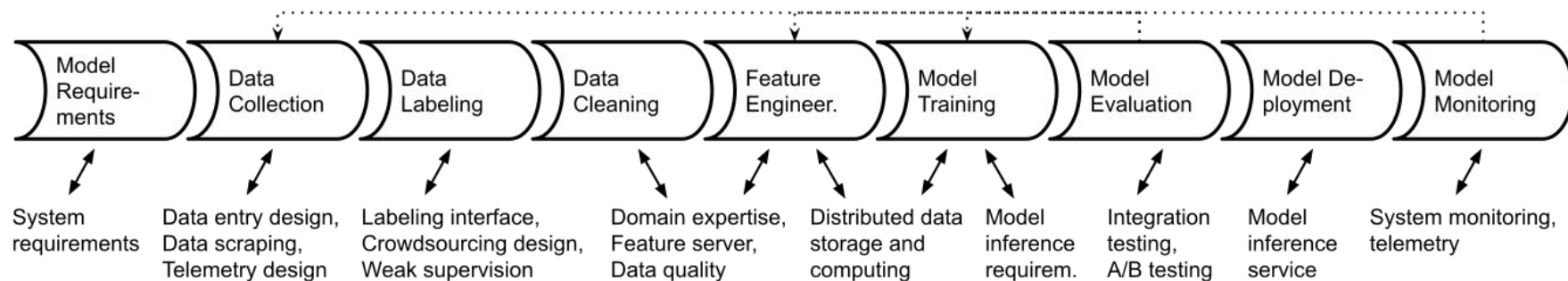
Monitoring Tests



1. Dependency changes result in notification.
2. Data invariants hold for inputs.
3. Training and serving are not skewed.
4. Models are not too stale.
5. Models are numerically stable.
6. Computing performance has not regressed.
7. Prediction quality has not regressed.

Testing across boundaries: Minimizing and Stubbing Dependencies

Real Pipelines can be Complex



How to unit test component with dependency on other code?

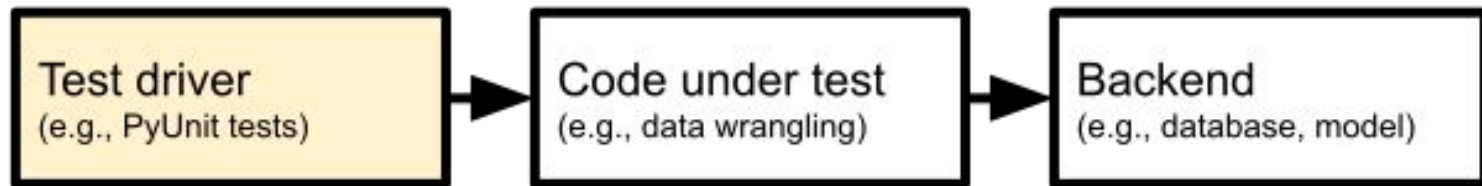


How to Test Parts of a System?



```
# original implementation hardcodes external API
def clean_gender(df):
    def clean(row):
        if pd.isnull(row['gender']):
            row['gender'] = gender_api_client.predict(
                row['firstname'], row['lastname'], row['location'])
    return row
return df.apply(clean, axis=1)
```

Automating Test Execution



```
def test_do_not_overwrite_gender():
    df = pd.DataFrame({'firstname': ['John', 'Jane', 'Jim'],
                       'lastname': ['Doe', 'Doe', 'Doe'],
                       'location': ['Pittsburgh, PA', 'Rome, Italy', 'Paris, PA '],
                       'gender': [np.nan, 'F', np.nan]})
    out = clean_gender(df, model_stub)
    assert(out['gender'] == ['M', 'F', 'M']).all()
```

Decoupling from Dependencies

```
def clean_gender_col(df, model):  
    def clean(row):  
        if pd.isnull(row['gender']):  
            row['gender'] = model(row['firstname'],  
                                  row['lastname'],  
                                  row['location'])  
    return row  
    return df.apply(clean, axis=1)
```

Replace concrete API with an interface that caller can parameterize

Stubbing the Dependency

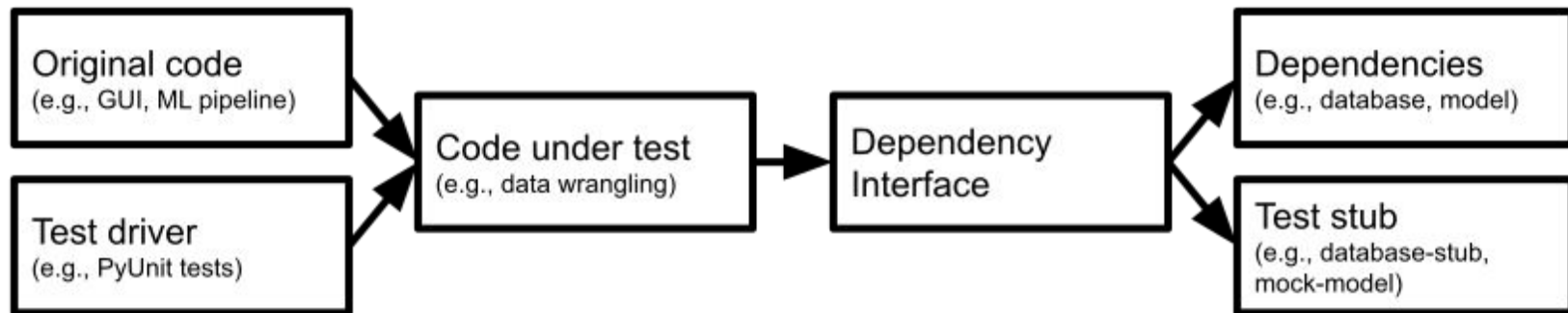


```
def test_do_not_overwrite_gender():
    def model_stub(first, last, location):
        return 'M'

    df = pd.DataFrame({'firstname': ['John', 'Jane', 'Jim'],
                       'lastname': ['Doe', 'Doe', 'Doe'],
                       'location': ['Pittsburgh, PA', 'Rome, Italy', 'Paris, PA '],
                       'gender': [np.nan, 'F', np.nan]})

    out = clean_gender(df, model_stub)
    assert(out['gender'] == ['M', 'F', 'M']).all()
```

General Testing Strategy: Decoupling Code Under Test



(Mocking frameworks provide infrastructure for expressing such tests compactly.)

Unit Tests with LLM Calls

Tests that call a live model are slow, expensive, hit rate limits, and are flaky

Temperature 0 alone does not make a test deterministic

- Inference output depends on the batch size of the server
- Providers update models behind the same model name

Flaky tests get ignored

Avoid real LLM calls in unit tests

Stubbing the Model

Hardcoded/handwritten LLM responses

```
class ScriptedLLM:
    def __init__(self, *replies):
        self.replies, self.prompts = list(replies), []
    def __call__(self, prompt):
        self.prompts.append(prompt)    # observability: what did the model see?
        return self.replies.pop(0)    # controllability: what does it answer?
```

Record and Replay Model Calls

Record real model replies once during test development, replay them in CI

Map recorded answer to specific request (tuple of model ID, prompt, parameters, tools)

If a recording is missing, fail the test

```
class ReplayLLM:
    def __init__(self, path):
        self.replies = json.loads(Path(path).read_text())

    def __call__(self, prompt):
        return self.replies[prompt]  # KeyError if the prompt changed

def test_recap_with_recorded_reply():
    llm = ReplayLLM("recordings/recap.json")
    recap = summarize_unread(MESSAGES, llm=llm)
    assert recap.message_ids == [3, 7]
```

Test Strategies for Code that Calls LLMs

Layer	Model replaced by	When	Oracle
Unit and harness tests	Stub or scripted fake	Every commit	Exact
Integration tests	Recorded real replies (replay)	Every commit	Exact
Live smoke tests	Real model API	Nightly or before release	Tolerant: valid schema, no errors, latency and cost limits
Evals (see Behavioral Model Testing)	Real model	On prompt or model change	Statistical threshold, LLM judge
Testing in production	Real model	During/after deployment	Telemetry (A/B or trend)

Breakout: Testing a Meeting Agent

Planning agent has access to GitHub issues where team tasks and progress is tracked. Agent generates meeting agendas and updates issue tracker after meeting based on transcript.

Test function `proccess_transcript(transcript)` for this: Inside this function, an LLM recognizes actions from meeting transcript (e.g., close #12, assign #5 to Ann, create new "..."). Must refuse to modify issues that do not exist.

As a group, post and follow up in #lecture, tagging group members and @mlipbot: *Write a (python-ish) test with stubs for this behavior*

Compound AI Systems

“Modern AI Pipelines”

Less focus on data wrangling, model training/fine tuning,
more on prompting (“context engineering”)

Multiple LLM calls

Integration with tools (retrieval, send message, ...)

Silent Mistakes in Compound AI Systems?

Recall:

Prompt templates

Sequences and routing of models/prompts

Retrieval augmented generation

Agents

Prompt optimization

Memory and harness optimization



Possible Mistakes in Compound AI Systems

- A prompt variable is empty or None, but the model still answers fluently
- Context truncation drops the system prompt or the newest messages
- Output stops at max_tokens: cut-off JSON, the parser returns a default value
- A tool fails, the model gets an empty result and invents an answer
- A retry after a timeout runs a side-effect tool two times (e.g., send email)
- RAG index is stale, or uses a different embedding model than the queries
- Retrieval filter bug returns documents of a different user
- Model alias moves to a new snapshot, or the model is retired; fallback runs silently
- Cache key without model or prompt version returns old answers
- Moderation service times out, the pipeline skips the check ("fails open")
- Telemetry without prompt version: cannot link cost or feedback to a change

Subtle Bugs in Context Assembly Code

```
context = history[:MAX_MESSAGES]           # keeps oldest, drops newest
```

```
messages = ([system] + history)[-MAX_MESSAGES:] # drops system prompt
```

```
Template("Recap for {{ user }}: {{ msgs }}").render(msgs=m)  
# Jinja2: missing variable -> ""
```

```
if len(prompt) > 8000: ...                   # characters, not tokens
```

```
try: recap = json.loads(reply.text)  
except json.JSONDecodeError: recap = {"summary": ""} # silent default
```

```
try: result = tool(**args)  
except Exception: result = ""               # model sees "no data"
```

Agents: Test the Harness, not the Model

Test correct context assembly

Test how the model processes tool calls

Test skill loading, memory updates, workflows

Test safeguards, hooks, sandboxes, subagents

Tests the tools

Test error handling

An Empirical Study of Testing Practices in Open Source AI Agent Frameworks and Agentic Applications. Empirical Software Engineering, 2026 (arXiv:2509.19185).

Machine Learning in Production • Bogdan Vasilescu & Christian Kaestner, Carnegie Mellon University • Fall 2026

Fail-Plausible: Silent Mistakes Get Worse

ML pipeline: a mistake gives wrong data, often with some visible symptom

LLM pipeline: the model turns an error into a fluent, plausible answer ("fail-plausible")

- "You have no meetings tomorrow" when the calendar API failed

409 fixed bugs in 5 agent frameworks:

- 76% of symptoms are wrong or missing results, not crashes
- The model-integration layer has the most bugs, but only 47% of its fixes add tests

Detect errors outside the model's text: error fields, counters, alerts

Tests assert on the state of the harness, not only on the answer

Wu. When Errors Become Narratives: silent failures in a production LLM agent runtime. arXiv:2606.14589, 2026 (preprint). | Understanding Bugs in Modern Agentic Frameworks. arXiv:2604.08906, 2026.

Breakout 2: Testing the Meeting Agent

Planning agent has access to GitHub issues where team tasks and progress is tracked. Agent generates meeting agendas and updates issue tracker after meeting based on transcript.

As a group, post and follow up in #lecture, tagging group members and @mlipbot:

- Identify 10 useful tests for the agent harness (identify what to test, no need to explain how or write code)

Real Infrastructure Bugs at an LLM Provider

Anthropic postmortem (Sept 2025): three overlapping bugs reduced answer quality for some users

- Routing bug: some requests went to servers configured for a different context window
- Server misconfiguration corrupted token generation (e.g., characters of the wrong language)
- Compiler bug in approximate top-k sampling

Silent mistakes, not model bugs, tests did not find them

Anthropic. A postmortem of three recent issues. Engineering blog, Sept 2025. <https://www.anthropic.com/engineering/a-postmortem-of-three-recent-issues>

Testing Error Handling / Infrastructure Robustness

General Error Handling Strategies

Avoid silent errors

Recover locally if possible, propagate error if necessary -- fail entire task if needed

Explicitly handle exceptional conditions and mistakes

Test correct error handling

If logging only, is anybody analyzing log files?

Test for Expected Exceptions

```
def test_learning_fails_with_missing_data():  
    df = pd.DataFrame({})  
    with pytest.raises(NoDataError):  
        learn(df)
```

Test Recovery Mechanisms with Stub

Use stubs to inject artificial faults

```
## testing retry mechanism
from retry.api import retry_call
import pytest

# stub of a network connection, sometimes failing
class FailedConnection(Connection):
    remaining_failures = 0
    def __init__(self, failures):
        self.remaining_failures = failures
    def get(self, url):
        print(self.remaining_failures)
        self.remaining_failures -= 1
        if self.remaining_failures >= 0:
            raise TimeoutError('fail')
        return "success"

# function to be tested, with recovery mechanism
def get_data(connection, value):
    def get(): return connection.get('https://replicate.npmjs.com/registry/'+value)
    return retry_call(get,
        exceptions = TimeoutError, tries=3, delay=0.1, backoff=2)

# 3 tests for no problem, recoverable problem, and not recoverable
def test_no_problem_case():
    connection = FailedConnection(0)
    assert get_data(connection, '') == 'success'

def test_successful_recovery():
    connection = FailedConnection(2)
    assert get_data(connection, '') == 'success'

def test_exception_if_unable_to_recover():
    connection = FailedConnection(10)
    with pytest.raises(TimeoutError):
        get_data(connection, '')
```

Test Error Handling throughout Pipeline

Is invalid data rejected / repaired?

Are missing data updates raising errors?

Are unavailable APIs triggering errors?

Are failing deployments reported?

Test Error Handling for Model APIs

Inject LLM-specific faults with stubs:

- rate limit (HTTP 429 with retry-after), overload (5xx), timeout
- context too long, output cut off at max_tokens, refusal
- model not found (retired model)

Test the recovery mechanisms:

- retry obeys retry-after and stops after N attempts
- fallback to a second model or provider; clear degraded answer
- token and cost budget per request
- a retry does not repeat a side-effect tool call (idempotency key)

Inject Model API Faults with a Stub

```
class FlakyModel:
    def __init__(self, failures):
        self.failures, self.calls = list(failures), 0
    def __call__(self, prompt):
        self.calls += 1
        if self.failures:
            raise self.failures.pop(0)
        return "ok"

def test_fallback_after_rate_limits():
    primary = FlakyModel([RateLimitError(retry_after=0)] * 3)
    backup = FlakyModel([])
    client = LLMClient([primary, backup], max_attempts=3)
    assert client.complete("hi") == "ok"
    assert primary.calls == 3 and backup.calls == 1

def test_no_duplicate_email_after_timeout():
    outbox = FakeOutbox(fail_first_ack=True) # email sent, ack times out
    send_with_retry(outbox, email, idempotency_key="req-42")
    assert len(outbox.sent) == 1
```

Log Error Occurrence

Even when reported or mitigated, log the issue

Allows later analysis of frequency and patterns

Monitoring systems can raise alarms for anomalies

Example: Error Logging

```
from prometheus_client import Counter
connection_timeout_counter = Counter(
    'connection_retry_total',
    'Retry attempts on failed connections')

class RetryLogger():
    def warning(self, fmt, error, delay):
        connection_timeout_counter.inc()

retry_logger = RetryLogger()

def get_data(connection, value):
    def get(): return connection.get('https://replicate.npmjs.com/registry/'+value)
    return retry_call(get,
        exceptions = TimeoutError, tries=3, delay=0.1, backoff=2,
        logger = retry_logger)
```

Test Monitoring

Inject/simulate
faulty behavior

Mock out
notification service
used by monitoring

Assert notification

```
class FakeNotifier:
    def __init__(self):
        self.messages = []
    def send(self, msg):
        self.messages.append(msg)

def test_alert_when_server_down():
    server = FakeServer()
    notifier = FakeNotifier()
    monitor = Monitor(server, notifier)
    server.stop()
    server.request(); server.request()
    monitor.check()          # or wait
    assert len(notifier.messages) == 1
```

Testing Error Handling in Compound AI System

```
def test_agent_handles_tool_failure():  
    """Agent should gracefully handle a tool that errors out."""  
    def failing_calendar(date_range):  
        raise ConnectionError("Calendar API unavailable")  
    result = react_agent("What meetings do I have tomorrow?",  
                        tools={"get_calendar": failing_calendar},  
                        max_steps=10, llm=fake_llm)  
    assert "unavailable" in result.answer.lower() or result.terminated
```

Test Monitoring in Production

Like fire drills (manual tests may be okay!)

Manual tests in production, repeat regularly

Actually take down service or trigger wrong signal to monitor

Where to Focus Testing?



Testing in ML Pipelines

Usually assume ML libraries already tested (pandas, sklearn, etc)

Focus on custom code

- data quality checks
- data wrangling (feature engineering)
- training setup
- interaction with other components

Consider tests of latency, throughput, memory, ...

Testing Data Quality Checks

Test correct detection of problems

```
def test_invalid_day_of_week_data():  
    ...
```

Test correct error handling or repair of detected problems

```
def test_fill_missing_gender():  
    ...  
def test_exception_for_missing_data():  
    ...
```

Test Data Wrangling Code

```
num = data.Size.replace(r'[kM]+$ ', '', regex=True).  
    astype(float)  
factor = data.Size.str.extract(r'[\d\.]+([KM]+)',  
                               expand=False)  
factor = factor.replace(['k', 'M'], [10**3, 10**6]).fillna(1)  
data['Size'] = num*factor.astype(int)
```

```
data["Size"] = data["Size"].  
    replace(regex=['k'], value='000')  
data["Size"] = data["Size"].  
    replace(regex=['M'], value='000000')  
data["Size"] = data["Size"].astype(str).astype(float)
```

Test Context Assembly and Result Parsing

```
def test_truncation_keeps_system_prompt_and_newest():
    history = [message(f"m{i}") for i in range(100)]
    messages = build_context(SYSTEM, history, token_budget=500)
    assert messages[0] == SYSTEM
    assert messages[-1] == history[-1]
    assert count_tokens(messages) <= 500

def test_recap_recovers_from_invalid_json():
    llm = ScriptedLLM('{"summary": "Standup moved to', # cut off
                      '{"summary": "Standup moved to 10am"}')
    recap = summarize_unread(MESSAGES, llm=llm)
    assert recap.summary == "Standup moved to 10am"
    assert len(llm.prompts) == 2 # retried once
    assert "invalid JSON" in llm.prompts[-1] # error sent back
```

Test Model Training Setup?

Execute training with small sample data

Ensure shape of model and data as expected (e.g., tensor dimensions)

Test Interactions with Other Components

Test error handling for detecting connection/data problems

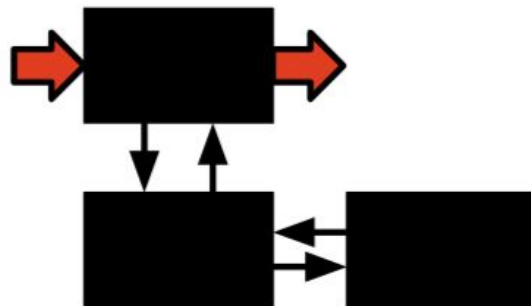
- loading training data
- feature server
- uploading serialized model
- A/B testing infrastructure

Integration and system tests

Unit test



Integration test



System Test



Integration and system tests

Test larger units of behavior
Often based on use cases or user stories -- from the customer's perspective

```
# Story: recap shows unread messages

def test_recap_lists_unread_only():
    llm = ReplayLLM("recap.json")
    app = TestClient(create_app(llm))
    app.post("/send", json=MSG_1)
    app.post("/send", json=MSG_2)
    app.post("/read", json={"id": 1})
    r = app.get("/recap")
    assert r.status_code == 200
    assert r.json()["ids"] == [2]
```

Integration tests

```
def test_cleaning_with_feature_eng():  
    d = load_test_data()  
    cd = clean(d)  
    f = feature3.encode(cd)  
    assert no_missing_values(f["m"])  
    assert max(f["m"]) <= 1.0
```

Test combined behavior of multiple functions

Test Integration of Components

```
// making predictions with an ensemble of models
function predict_price(data, models, timeouts) {
  // send asynchronous REST requests all models
  const requests = models.map(model => rpc(model, data, {timeout: timeouts})).
    then(parseResult).catch(e => -1))
  // collect all answers and return average if at least two models succeeded
  return Promise.all(requests).then(predictions => {
    const success = predictions.filter(v => v >= 0)
    if (success.length < 2) throw new Error("Too many models failed")
    return success.reduce((a, b) => a + b, 0) / success.length
  })
}

// test ensemble of models
const timeout = 500, M1 = "http://localhost:3000/predict", ...
beforeAll(() => {
  // launch model 1 API at address M1
  // launch model 2 API at address M2
  // launch model API with timeout at address M3
})
afterAll(() => { /* shut down all model APIs */ })

test("success despite timeout", async () => {
  const start = performance.now();
  const val = await predict_price(input, [M1, M2, M3], timeout)
  expect(performance.now() - start).toBeLessThan(2 * timeout)
  expect(val).toBeGreaterThan(0)
})

test("fail on too many timeouts", async () => {
  const start = performance.now();
  await expect(predict_price(input, [M1, M3, M3], timeout))
    .rejects.toThrow("Too many models failed")
})
```

End-To-End Test of Entire Pipeline

```
def test_pipeline():  
    train = pd.read_csv('pipelinetest_training.csv', parse_dates=True)  
    test = pd.read_csv('pipelinetest_test.csv', parse_dates=True)  
    X_train, y_train = prepare_data(train)  
    X_test, y_test = prepare_data(test)  
    model = learn(X_train, y_train)  
    accuracy = eval(model, X_test, y_test)  
    assert accuracy > 0.9
```

System Testing from a User Perspective

Test the product as a whole, not just components

Click through user interface, achieve task
(often manually performed)

Derived from requirements (use cases, user stories)

Testing in production

Bonus: Data Linter at Google

Miscoding

- Number, date, time as string
- Enum as real
- Tokenizable string (long strings, all unique)
- Zip code as number

Outliers and scaling

- Unnormalized feature (varies widely)
- Tailed distributions
- Uncommon sign

Packaging

- Duplicate rows
- Empty/missing data

Further readings: Hynes, Nick, D. Sculley, and Michael Terry. [The data linter: Lightweight, automated sanity checking for ML data sets](#). NIPS MLSys Workshop. 2017.

Summary

Summary

- Beyond model and data quality: Quality of the infrastructure matters, danger of silent mistakes
- Automate pipelines to foster testing, evolution, and experimentation
- Many SE techniques for test automation, testing robustness, test adequacy, testing in production useful for infrastructure quality

Further Readings

- O'Leary, Katie, and Makoto Uchida. "[Common problems with Creating Machine Learning Pipelines from Existing Code](#)." Proc. Third Conference on Machine Learning and Systems (MLSys) (2020).
- Eric Breck, Shanjing Cai, Eric Nielsen, Michael Salib, D. Sculley. The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction. Proceedings of IEEE Big Data (2017)
- Zinkevich, Martin. [Rules of Machine Learning: Best Practices for ML Engineering](#). Google Blog Post, 2017
- Serban, Alex, Koen van der Blom, Holger Hoos, and Joost Visser. "[Adoption and Effects of Software Engineering Best Practices in Machine Learning](#)." In Proc. ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (2020).